

A Technical Introduction to Momentum optimization

Section 1

Backtracking line search Backtracking line search is another variant of gradient descent. All of the below are sourced from the mentioned link. It is based on a condition known as the Armijo–Goldstein condition. Both methods allow learning rates to change at each iteration; however, the manner of the change is different. Backtracking line search uses function evaluations to check Armijo's condition, and in principle the loop in the algorithm for determining the learning rates can be long and unknown in advance. Adaptive SGD does not need a loop in determining learning rates. On the other hand, adaptive SGD does not guarantee the "descent property" – which Backtracking line search enjoys – which is that $f(x_{n+1}) \leq f(x_n)$ for all n . If the gradient of the cost function is globally Lipschitz continuous, with Lipschitz constant L , and learning rate is chosen of the order $1/L$, then the standard version of SGD is a special case of backtracking line search.

Section 2

where, γ is the forgetting factor. The concept of storing the historical gradient as sum of squares is borrowed from Adagrad, but "forgetting" is introduced to solve Adagrad's diminishing learning rates in non-convex problems by gradually decreasing the influence of old data. And the parameters are updated as,

Section 3

AdaGrad AdaGrad (for adaptive gradient algorithm) is a modified stochastic gradient descent algorithm with per-parameter learning rate, first published in 2011. Informally, this increases the learning rate for sparser parameters and decreases the learning rate for ones that are less sparse. This strategy often improves convergence performance over standard stochastic gradient descent in settings where data is sparse and sparse parameters are more informative. Examples of such applications include natural language processing and image recognition. It still has a base learning rate η , but this is multiplied with the elements of a vector $\{G_{j,i}\}$ which is the diagonal of the outer product matrix

Section 4

Further proposals include the momentum method or the heavy ball method, which in ML context appeared in Rumelhart, Hinton and Williams' paper on backpropagation learning and borrowed the idea from Soviet mathematician Boris Polyak's 1964 article on solving functional equations. Stochastic gradient descent with momentum remembers the update Δw at each iteration, and determines the next

update as a linear combination of the gradient and the previous update:

Section 5

$$dW_t = -\nabla(Q(W_t) + \frac{1}{4}\eta|\nabla Q(W_t)|^2)dt + \eta\Sigma(W_t)^{1/2}dB_t, \quad \left\{ \begin{array}{l} \nabla(Q(W_t) + \frac{1}{4}\eta|\nabla Q(W_t)|^2) \\ \Sigma(W_t)^{1/2} \end{array} \right\}$$

Section 6

subject to additional stochastic noise. This approximation is only valid on a finite time-horizon in the following sense: assume that all the coefficients Q_i are sufficiently smooth. Let $T > 0$ and $g: \mathbb{R}^d \rightarrow \mathbb{R}$ be a sufficiently smooth test function. Then, there exists a constant $C > 0$ such that for all $\eta > 0$

Section 7

$\bar{w} = \frac{1}{t} \sum_{i=0}^{t-1} w_i$. When optimization is done, this averaged parameter vector takes the place of w .

Section 8

Adam Adam (short for Adaptive Moment Estimation) is a 2014 update to the RMSProp optimizer combining it with the main feature of the Momentum method. In this optimization algorithm, running averages with exponential forgetting of both the gradients and the second moments of the gradients are used. Given parameters $w(t)$ and a loss function $L(t)$, where t indexes the current training iteration (indexed at 1), Adam's parameter update is given by:

Section 9

where i is uniformly sampled between 1 and n . Although theoretical convergence of this procedure happens under relatively mild assumptions, in practice the procedure can be quite unstable. In particular, when η is misspecified so that $I - \eta x_i x_i'$ has large absolute eigenvalues with high probability, the procedure may diverge numerically within a few iterations. In contrast, implicit stochastic gradient descent (shortened as ISGD) can be solved in closed-form as:

Section 10

Second-order methods A stochastic analogue of the standard (deterministic) Newton–Raphson algorithm (a "second-order" method) provides an asymptotically optimal or near-optimal form of iterative optimization in the setting of stochastic approximation. A method that uses direct

measurements of the Hessian matrices of the summands in the empirical risk function was developed by Byrd, Hansen, Nocedal, and Singer. However, directly determining the required Hessian matrices for optimization may not be possible in practice. Practical and theoretically sound methods for second-order versions of SGD that do not require direct Hessian information are given by Spall and others. (A less efficient method based on finite differences, instead of simultaneous perturbations, is given by Ruppert.) Another approach to the approximation Hessian matrix is replacing it with the Fisher information matrix, which transforms usual gradient to natural. These methods not requiring direct Hessian information are based on either values of the summands in the above empirical risk function or values of the gradients of the summands (i.e., the SGD inputs). In particular, second-order optimality is asymptotically achievable without direct calculation of the Hessian matrices of the summands in the empirical risk function. When the objective is a nonlinear least-squares loss

Section 11

$Q(w) = \frac{1}{n} \sum_{i=1}^n Q_i(w) = \frac{1}{n} \sum_{i=1}^n (m(w; x_i) - y_i)^2$, $\{ \displaystyle Q(w) = \frac{1}{n} \sum_{i=1}^n Q_i(w) = \frac{1}{n} \sum_{i=1}^n (m(w; x_i) - y_i)^2 \}$, where $m(w; x_i)$ $\{ \displaystyle m(w; x_i) \}$ is the predictive model (e.g., a deep neural network) the objective's structure can be exploited to estimate 2nd order information using gradients only. The resulting methods are simple and often effective